

Tree Generation Parameter Estimation from Point Clouds

Anej Umek¹

¹Faculty of Computer and Information Science, University of Ljubljana, Slovenia

Abstract

3D scans of the environment have become more and more common in the recent years with drones, UAVs and autonomous cars becoming more accessible. But the generated point clouds are not interpretable so they are usually transformed into some other type of data. One particular area of reconstruction is the reconstruction of vegetation or trees. In this paper I tackle the reconstruction of trees from an arial scan. I create a synthetic dataset of trees using Blender and simulate an aerial LiDAR scan to create their point clouds. After the creation of the dataset, PointNet++ is used to predict the most important parameters of the trees which can be used for their reconstruction. I also create a simple Blender add-on to test the capabilities of the model. On test set the model achieves great results with 98.0% accuracy on the classification parameter, while the qualitative results show that the reconstruction of the trees from the point cloud creates similar looking trees to the original.

1. Introduction

Converting 3D point clouds of trees into their corresponding tree models has many good applications such as environment scan compression (compressing whole point clouds of forests into a handful of useful parameters), biomass estimation, forest management/study etc. But the task is challenging because trees greatly vary in shapes and sizes and are often densely packed together, which makes the task even harder since LiDAR scans are sparse and it is challenging to distinguish which data point is part of which tree.

In this paper I only tackle the problem of reconstructing individual trees by creating a dataset and training a deep learning model to estimate the parameters. The code together with the dataset is available [here](#).

2. Related work

X. Wang et al. [WXN*23] tackled tree generation by simulating their natural growth. They use oblique photogrammetry-supporting procedural tree modeling approach for which they use control-parameter analysis to find the solution that maximizes resemblance to the input tree point cloud.

Lowe et al. [LP22] had a different goal. They wanted to extract the branch structure of the trees given the point cloud data which they achieved by methods of structural topology optimisation as a dense reconstruction, which resulted in efficient distribution of branches that matches the input point cloud.

S. Hu et al. [HLZ*17] were tasked with modeling trees of the whole forest which they solved by first segmenting the forest point cloud into individual trees, constructing a connected graph using

nearest neighbors, calculating a direction field and finally creating a tree skeleton to which they added branch geometry and leaves according to botanical rules.

3. Methodology

3.1. Dataset creation

To create a dataset, I used Blender 3D graphics software tool. The trees were created with the Modular Tree add-on [BB26] which uses a node-based workflow for procedural tree generation. A simple workflow example is presented in figure 1.

Next, a geometry node setup was created to simulate LiDAR aerial scan and create a point cloud of the tree. A grid of points spaced 20 centimeters apart above the tree are projected straight down (using Raycast node) and if they hit the tree they stay at that position, else they are removed. The geometry node workflow can be seen in figure 2 and an example of the random generated tree and a point cloud can be seen in figure 3.

The tree and point cloud generation explained above was then written in Blender Python API, which allowed us to create a dataset. Most of the tree parameters are fixed, since they don't change the general shape and size of the tree much, the only changing parameters are:

- **trunk length** - height of the trunk,
- **trunk radius** - the radius of the trunk at the bottom,
- **branch length** - length of the braches from the trunk,
- **branch density** - how densly the branches populate the trunk,
- **branch up attraction** - how do the branches curve (up/down) away from the trunk,
- **crown shape** - the shape of the tree crown, can be one of the

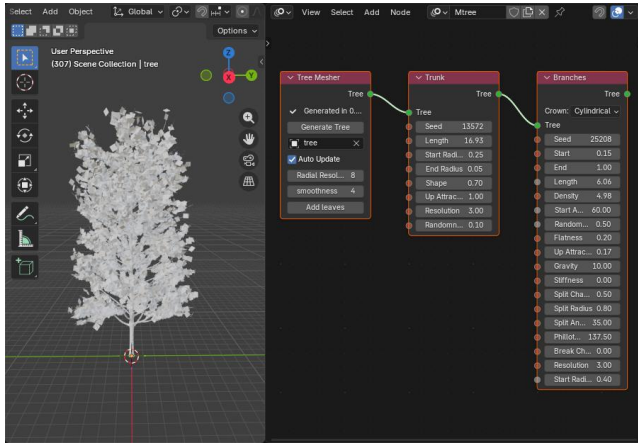


Figure 1: An example of the Modular tree node workflow in the right window and the corresponding generated tree in the left window.

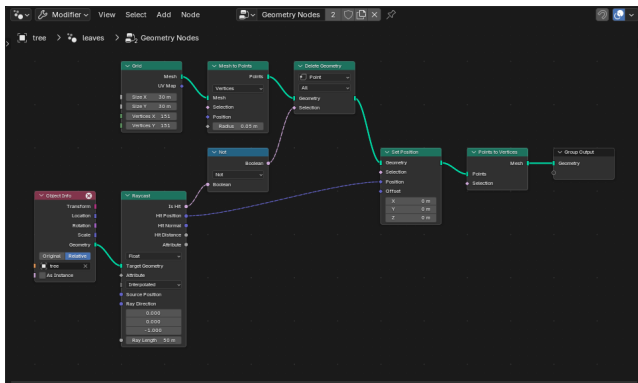


Figure 2: Geometry nodes setup for simulating LiDAR scan and creating a point cloud.

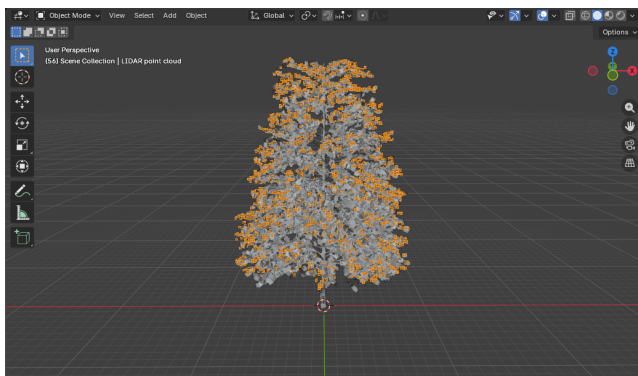


Figure 3: An example of a randomly generated tree and the generated point cloud (orange dots).

7 shapes (cylindrical, conical, spherical, hemispherical, tapered cylindrical, inverse conical or tend flame).

The first 5 parameters are regression parameters while the last one is a classification parameter. The script was then used to generate 5000 point clouds and save them together with the above presented parameters.

3.2. Model

For the deep learning model I picked PointNet++, which is an extension of the original PointNet model. PointNet [QSMG17] was a pioneer in deep models that performed deep learning task such as classification and part segmentation directly on points of the point cloud rather than transforming those points to voxel grid or collection of images. To achieve permutation invariance, each point is processed independently (by shared multi-layer perceptrons) and then a symmetrical function (max pool) combines the point feature vector information into one global feature vector which can then be used for classification. To achieve invariance to rigid transformation, they apply a spatial alignment transformation of the input points by transforming them into canonical pose.

While the original PointNet performed well on many dataset, it had a big drawback by not considering the local structure and neighborhood of the points. PointNet++ [QYSG17] addresses this issue by processing points hierarchically. It samples points using farthest point sampling, which become centroids. Each centroid then finds nearby points and creates a local neighborhood on which the original PointNet is applied so that it creates a feature vector of the local neighborhood. This process is then repeated on the generated local features for several layers, where feature vectors of larger and larger neighborhoods merge together. This is similar process to CNN-based architecture where earlier layer have small receptive field while later layers have a large receptive field. This improvement as well as some other allowed PointNet++ to achieve much greater results compared to the original.

I used implementation of the PointNet++ that I have found online [Yan19] and modified the model by splitting the end into 2 heads, regression head for regression variables and classification head for the crown shape, which is the only classification variable. The total loss is calculated as a weighted sum of the regression loss (MSE) and classification loss (cross-entropy).

The model was trained with Adam optimizer, learning rate 0.001, weight decay 0.0001, betas 0.9 and 0.999, exponential learning rate scheduler with gamma 0.95 for 100 epochs using 2048 sampled points per point cloud. Regression loss weight was set to 0.25, while classification loss weight stayed at 1. This made the losses comparable in size.

80% of data (4000 point clouds) was used for training, while validation and test set each had 10% (500 point clouds each).

4. Results

In table 1 we can see the mean absolute error (MAE) results of each regression parameter on the test set. The results are promising with about 1.4 meters average error in the height of the tree trunk (trunk

	trunk		braches		
	length	radius	length	density	up attraction
MAE	1.402m	0.032m	0.517m	0.418	0.128

Table 1: MAE score for different regression parameters on the test set. Length and radius parameters have a unit of meters.

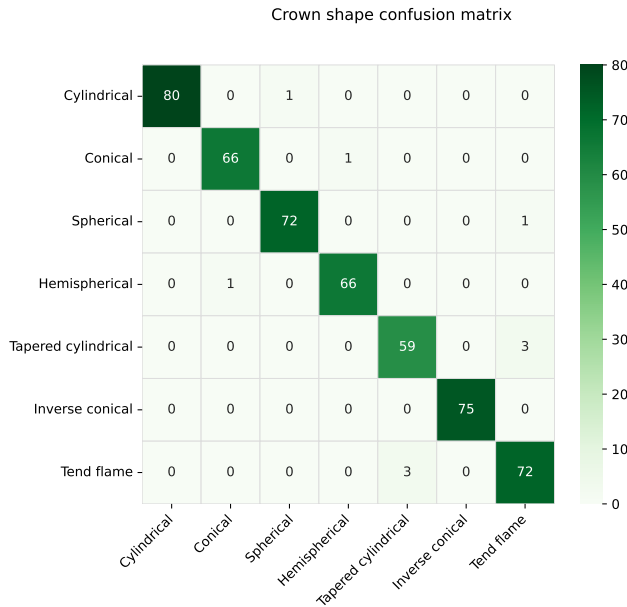


Figure 4: Confusion matrix for classification parameter (crown shape) on the test set. Rows represent the true labels while columns represent the predicted labels. The value in the cell represent the number of trees that were classified with the corresponding true and predicted labels.

is between 8 and 40 meters high). The radius has an average error of only 3 centimeters, while branches were usually off by about 0.5 meters.

In figure 4 we can see the confusion matrix of the classification parameter - crown shape. We can observe that the model predicts most of the crown shape correctly (98.0% classification accuracy). The model had the most problem distinguishing between the tapered cylindrical and tend flame crown shape with 3 wrong classifications (both ways).

Qualitative results with examples of the generated trees are presented in the appendix at the bottom.

5. Add-on

To show the capabilities of the learned model, a Blender add-on was created, which can be installed and accessed through the 3D view sidebar, with the name *Tree Parameter Estimation*. Clicking on a button reveals a a small panel interface with only 2 buttons:

- *Create Point Cloud* - creates a random tree and the corresponding point cloud,

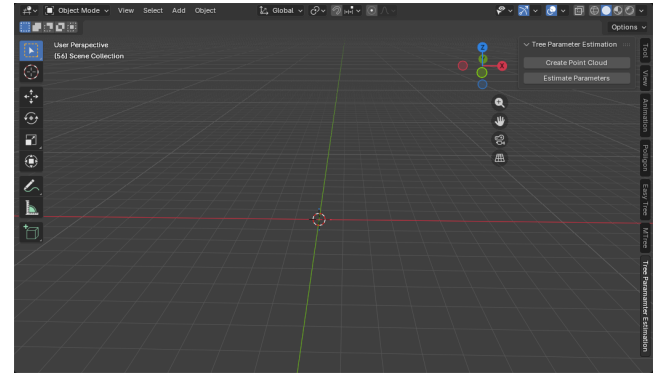


Figure 5: An example of the add-on UI on the right side of the image. Pressing the button in the sidebar shows a simple panel with 2 buttons.

- *Estimate Parameters* - estimates parameters of the active point cloud and create the tree at that position.

We can see how the UI of the add-on looks in figure 5.

6. Conclusion

In this paper I created a dataset of tree point clouds and their parameters and trained a deep neural network called PointNet++ to estimate the parameters of the tree given the point cloud. The model gives good results with 98.0% classification accuracy and low MAE on the regression parameters. I also created an add-on where random tree point clouds can be generated and tree parameters estimated which produces trees accurately and efficiently.

References

- [BB26] BRANDYN-BRITTON: Modular tree, 2026. URL: <https://extensions.blender.org/add-ons/modular-tree/>. 1
- [HLZ*17] HU S., LI Z., ZHANG Z., HE D., WIMMER M.: Efficient tree modeling from airborne lidar point clouds. *Computers & Graphics* 67 (2017), 1–13. URL: <https://www.sciencedirect.com/science/article/pii/S0097849317300420>, doi:<https://doi.org/10.1016/j.cag.2017.04.004>. 1
- [LP22] LOWE T., PINSKIER J.: Tree reconstruction using topology optimisation. *Remote Sensing* 15, 1 (2022), 172. 1
- [QSMG17] QI C. R., SU H., MO K., GUIBAS L. J.: Pointnet: Deep learning on point sets for 3d classification and segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (July 2017). 2
- [QYSG17] QI C. R., YI L., SU H., GUIBAS L.: Pointnet++: Deep hierarchical feature learning on point sets in a metric space. In *Advances in Neural Information Processing Systems* (2017), Guyon I., Luxburg U. V., Bengio S., Wallach H., Fergus R., Vishwanathan S., Garnett R., (Eds.), vol. 30, Curran Associates, Inc. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/d8bf84be3800d12f74d8b05e9b89836f-Paper.pdf. 2
- [WXN*23] WANG X., XIANG H., NIU W., MAO Z., HUANG X., ZHANG F.: Oblique photogrammetry supporting procedural tree modeling in urban areas. *ISPRS Journal of Photogrammetry and Remote Sensing* 200 (2023), 120–137. URL: <https://www.sciencedirect.com/science/article/pii/S0924271623001259>, doi:

<https://doi.org/10.1016/j.isprsjprs.2023.05.008>.
1

[Yan19] YAN X.: Pointnet/pointnet++ pytorch, 2019. URL: https://github.com/yanx27/Pointnet_Pointnet2_pytorch. 2

Appendix A: Qualitative results

Below are 2 figures, figure 6 and 7 which show examples of different generated trees, their corresponding point clouds and then generated trees. The trees vary in height, length, density and crown shape and we can see that the model generates trees that match up with the parameters of the original and creates a similar looking tree.

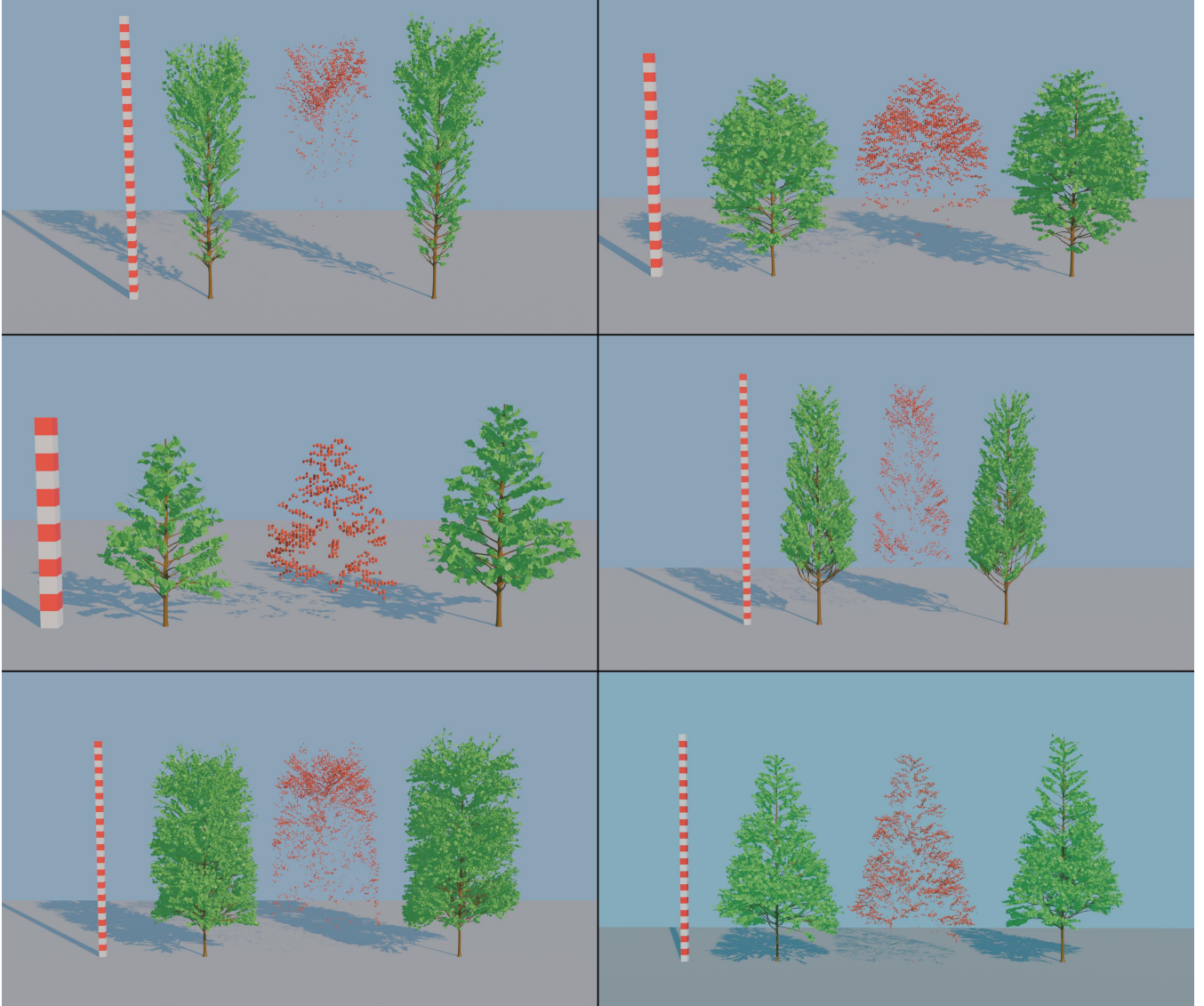


Figure 6: Qualitative examples 1 - 6 image examples with the left tree being the original, in the middle is the created point cloud and on the right is the tree that was generated by the model from the point cloud. On the left if also a tower of red and white cubes which shows the scale of the trees. Each cube has a length of 1 meter.

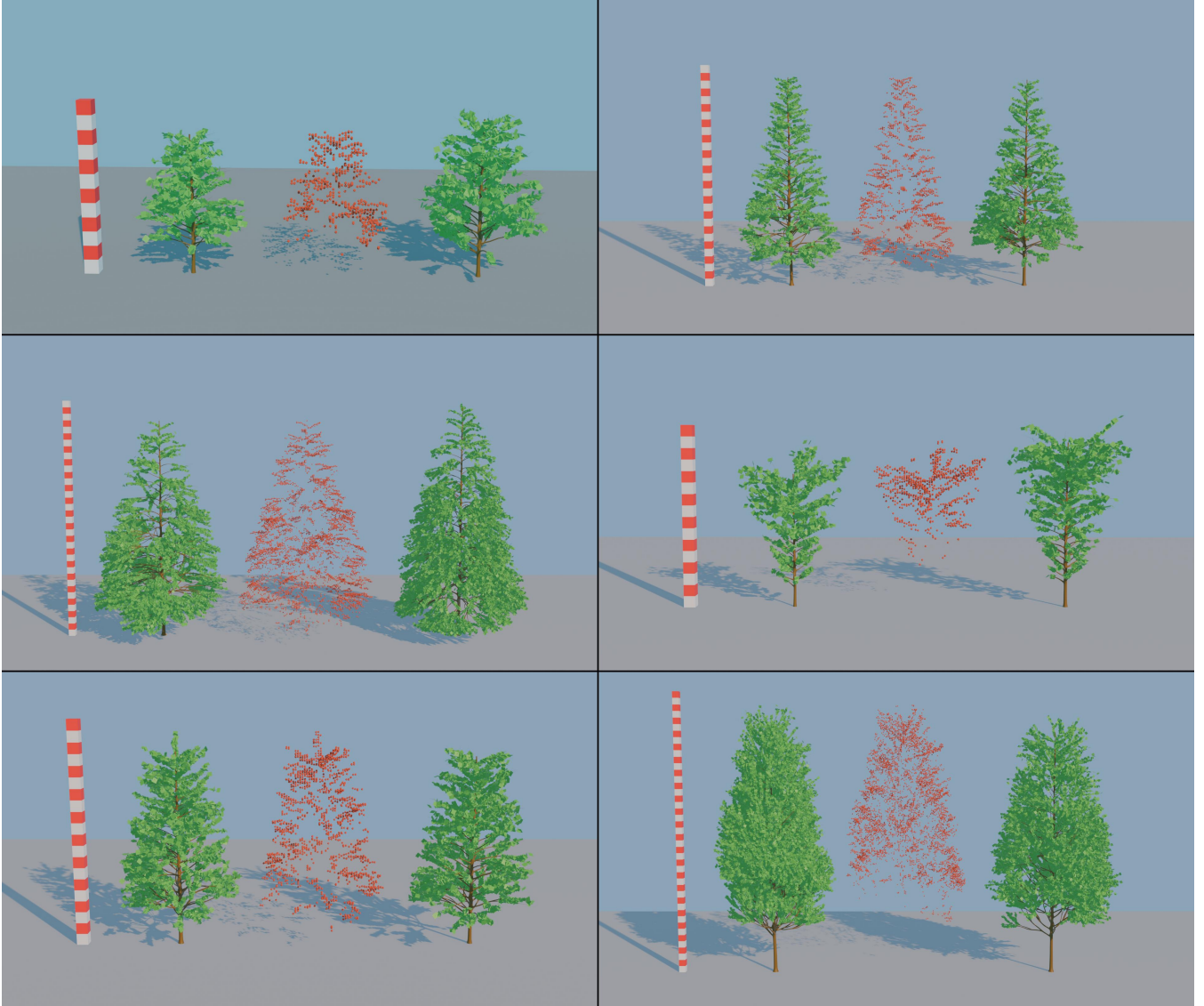


Figure 7: Qualitative examples 2 - 6 image examples with the left tree being the original, in the middle is the created point cloud and on the right is the tree that was generated by the model from the point cloud. On the left if also a tower of red and white cubes which shows the scale of the trees. Each cube has a length of 1 meter.